SUDOKU 6×6

Simple Team Development Plan • Java + JavaFX • 3 students

1. Project goal

Build a beginner-friendly 6×6 Sudoku game using Java and JavaFX. The goal is to meet the assignment requirements without adding unnecessary complexity. The board uses numbers 1–6, and every row, column, and 2×3 block must contain numbers without repetition.

2. Requirements from the assignment PDF

Requirement	Simple way to implement it
6×6 Sudoku	A grid with 6 rows and 6 columns; use numbers 1–6.
Java + JavaFX	Build the game in Java and display it with JavaFX.
Interface or adapter	Create one small Java interface, such as BoardValidator.
Data structure	Use a two-dimensional array: int[6][6].
MVC architecture	Separate the code into model, view, and controller packages.
Mouse and keyboard	Click a cell, type 1–6, and use Delete/Backspace to clear it.
JavaFX layouts	Use simple layouts such as GridPane, VBox, and HBox.
Javadoc	Add short documentation to important public classes and methods.
New game	Ask for confirmation, then show a valid board with 2 starting numbers in each 2×3 block.
Validation	Check rows, columns, and 2×3 blocks; show invalid entries clearly.
Help	Suggest one valid number in an empty cell; do not reveal the full solution.
GitHub	Show at least 2 commits per student and at least 2 pull requests per student as a safe team target.

3. Divide the work between three students

Student 1 — Sudoku logic (Model)

Main job: Make the Sudoku rules work without worrying about the graphical interface.
Tasks: represent the board with int[6][6]; check rows, columns, and 2×3 blocks; create a valid starting board; ensure each 2×3 block starts with exactly two given numbers; test valid and invalid moves.
Suggested files: SudokuBoard.java, SudokuValidator.java, SudokuGenerator.java, BoardValidator.java.
Deliverable: a working board and validation logic that can be called by the other students.

Student 2 — JavaFX interface (View)

Main job: Build the screen and let the player interact with the grid.
Tasks: create the 6×6 grid with GridPane; make cells selectable; accept keyboard numbers 1–6; allow Delete/Backspace; visually distinguish the 2×3 blocks and fixed starting numbers; add clear labels and buttons.
Suggested files: SudokuView.java, SudokuCell.java, MainApplication.java (or the project's existing JavaFX entry point).
Deliverable: a usable JavaFX screen with mouse and keyboard interaction.

Student 3 — Controller and help (Controller / Integration)

Main job: Connect the screen to the Sudoku logic and coordinate the game actions.
Tasks: create the controller; connect cell input to the model; show a confirmation before a new game; display validation errors and highlight invalid cells; implement a help button that suggests one valid number in an empty cell; help integrate and test the complete application.
Suggested files: SudokuController.java, SudokuHintService.java.
Deliverable: a connected game where the interface, rules, and help feature work together.

4. Keep the architecture simple (MVC)

Part	Responsible student	What it does
Model	Student 1	Stores the 6×6 board and checks Sudoku rules.
View	Student 2	Displays the grid, buttons, and user feedback.
Controller	Student 3	Responds to user actions and connects View with Model.

How they connect: The player interacts with the View. The Controller receives that action, asks the Model to validate or update the board, and then tells the View what to display.

5. Interface requirement — keep it small

Use one simple Java interface to satisfy the interface requirement. Student 1 can create BoardValidator and implement it in SudokuValidator. This is enough; do not add interfaces or adapter classes just for decoration.

Example:

```
public interface BoardValidator {
  boolean isValidMove(int row, int col, int number);
  boolean isValidBoard();
}
```

6. Keep the implementation easy

• Use int[6][6] for the board. Use 0 to represent an empty cell.

• Use ordinary for loops and simple methods to validate rows, columns, and 2×3 blocks.

• For the first version, use a predefined valid completed board and remove values to create the puzzle. This avoids a complicated puzzle-generation algorithm.

• Avoid databases, servers, multithreading, complex algorithms, and unnecessary design patterns.

• Do not create extra classes unless they have a clear job.

• Every student should understand and be able to explain the code they contribute.

7. GitHub workflow

Each student should work on a separate feature branch, push their own commits, and open pull requests into main. As a safe target, aim for **2 meaningful commits and 2 pull requests per student** (6 PRs total). The assignment requires evidence of at least 2 pull requests and 2 commits per student; do not rely on only two PRs for the whole team.

Student	Pull request 1	Pull request 2
1	Board structure	Generator and validator
2	JavaFX grid and layout	Mouse and keyboard interaction
3	Controller integration	Help and final integration

Example commands (run from the project folder):

```
git switch -c feature/sudoku-model
git add .
git commit -m "Create 6x6 Sudoku board structure"
git push -u origin feature/sudoku-model
```

Then open a Pull Request on GitHub. Use meaningful commits and PRs that contain real, reviewable work. Each student should use their own GitHub account so contributions are visible.

8. Suggested development order

1. Agree first: Agree on package names, class names, method names, and use 0 for empty cells.

2. Work separately: Student 1 builds the model; Student 2 builds the JavaFX grid; Student 3 prepares the controller structure and coordinates integration.

3. Connect the parts: Connect View → Controller → Model and test entering a number.

4. Finish the features: Add new-game confirmation, live validation, error highlighting, and help.

5. Test and document: Test all five user stories, add Javadoc, and prepare the README.

9. Final checklist

■ 6×6 grid displays correctly.

■ Each row, column, and 2×3 block has no repeated number.

■ New game asks for confirmation.

■ New board has exactly two starting numbers in each 2×3 block.

■ Player can select a cell and enter only 1–6.

■ Player can clear an entered number.

■ Invalid entries are detected and highlighted in real time.

■ Help suggests a valid number in an empty cell without solving the board.

■ Mouse and keyboard events work.

■ At least one interface and one data structure are implemented.

■ MVC packages are separated and understandable.

■ Important public classes and methods have Javadoc.

■ Each student has at least 2 commits and 2 PRs visible on GitHub.

10. Defense reminder

The assignment says the defense includes a code change and questions from the professor. The change is limited to 15 minutes. Every student must understand their own work and the overall flow. Do not submit AI-generated code that you cannot explain. Practice making a small change before the defense.

> **Guiding principle: Build the simplest version that satisfies the assignment. Avoid extra features until all required features work.**